

Spécifications techniques

Architecture, stack, modèle IA, API, sécurité, données, CI/CD, monitoring et déploiement de la plateforme de détection de maladies de la tomate.

Candidat	Sullivan Moreau
Certification	RNCP 37827 — Développeur en intelligence artificielle
Organisme	Simplon — promotion 2026 (projet individuel)
Date	Juillet 2026
Version	1.0 — document régénéré depuis docs/modelisation.md (source de vérité)

Sommaire

1. Vue d'ensemble de l'architecture
2. Stack technique justifiée
3. Modèle IA
4. API REST
5. Sécurité
6. Données
7. CI/CD
8. Monitoring et journalisation
9. Déploiement

1. Vue d'ensemble de l'architecture

L'application est composée de **5 services docker-compose** : api, front, postgres, prometheus, grafana. Le front Streamlit est **découplé du ML** (aucune dépendance torch côté front) : toute l'inférence passe par l'API. Le checkpoint du modèle (.pt) est monté en **volume Docker** en lecture seule, jamais embarqué dans l'image.

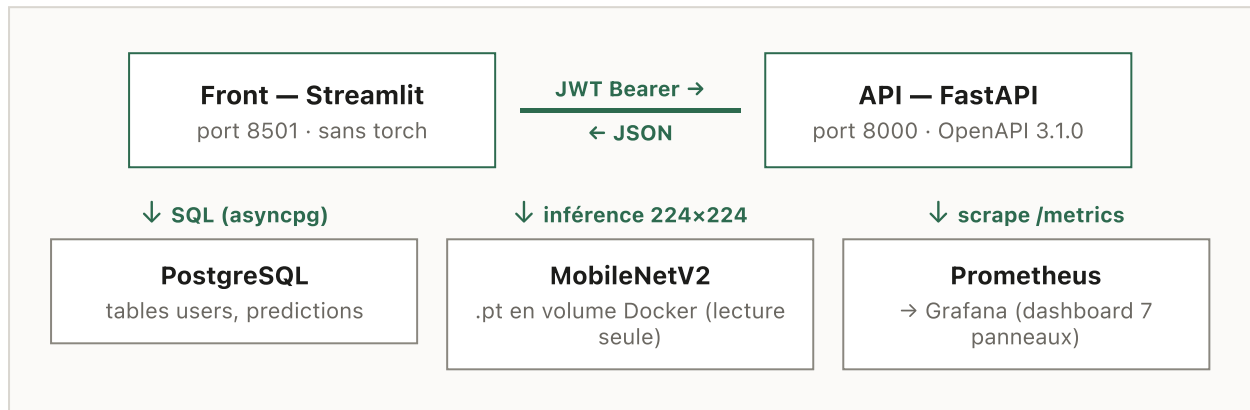


Figure 1 — Architecture des 5 services. Description textuelle : le front Streamlit appelle l'API FastAPI en HTTP (JWT Bearer, réponses JSON) ; l'API lit/écrit dans PostgreSQL, charge le checkpoint MobileNetV2 depuis un volume Docker pour servir les prédictions, et expose /metrics à Prometheus, visualisé dans Grafana.

Source Mermaid équivalente (versionnée avec le code) :

```
flowchart LR
    FRONT["Front — Streamlit (8501)"] -- "JWT Bearer / JSON" --> API["API — FastAPI (8000)"]
    API -- "SQL (asyncpg)" --> DB["PostgreSQL — users, predictions"]
    API -- "inférence 224x224" --> MODEL["MobileNetV2 — .pt en volume Docker"]
    API -- "GET /metrics" --> PROM["Prometheus"] --> GRAF["Grafana"]
```

2. Stack technique justifiée

Choix	Justification
PyTorch + MobileNetV2 torchvision, transfer learning	Architecture légère (~2,2 M paramètres) conçue pour l'inférence sur CPU, idéale pour un VPS sans GPU tout en gardant une excellente accuracy via le transfer learning ImageNet.
FastAPI	Framework Python asynchrone performant qui génère automatiquement la documentation OpenAPI 3.1.0 et valide les entrées via Pydantic.
Streamlit	Permet de livrer rapidement une interface web en pur Python, découplée du ML (aucun torch côté front, image Docker légère).
PostgreSQL async + Alembic asyncpg, SQLAlchemy	Accès concurrent réaliste sous charge (moteur asynchrone) et migrations de schéma versionnées et rejouables ; SQLite reste réservé au dev/préprod léger.
Prometheus + Grafana + loguru	Standard open source du monitoring : métriques applicatives et de comportement du modèle séparées, visualisées dans un dashboard, avec journalisation structurée.
Docker + Coolify sur VPS OVH images ghcr.io	Environnements reproductibles et déploiement automatisé auto-hébergé sur un VPS souverain (OVH Roubaix), sans dépendance à un PaaS propriétaire.

3. Modèle IA

Classification d'images par **transfer learning** sur **MobileNetV2** (framework **PyTorch** / torchvision) : la base convolutive pré-entraînée ImageNet est **gelée** ; seul le classifieur final est réentraîné — une couche Dropout suivie d'une couche Linear 1280 → 10.

Paramètres totaux	~2,2 M, dont ~12 810 entraînables (base gelée)
Dataset	PlantVillage — 10 classes de tomate (9 maladies + saine)
Accuracy test	93,55 % (2 402 images de test)
Accuracy validation	94,55 % — meilleure epoch : 13
F1 minimal par classe	0,806 — « Alternariose précoce » (Tomato_Early_blight)
Taille d'entrée	224 × 224 px

Gestion du checkpoint — pas de MLflow. Le modèle est un fichier .pt (poids model_state_dict + liste class_names), chargé **une seule fois en mémoire au démarrage de l'API** (singleton initialiser_modele(), chemin surchargé par MODEL_PATH). Il n'existe **ni table « modèle » en base, ni suivi d'expériences MLflow** : les rapports d'entraînement/évaluation sont des artefacts JSON/CSV versionnés dans docs/.

4. API REST

API FastAPI documentée en **OpenAPI 3.1.0** — **12 opérations REST sur 11 chemins**, plus le montage `/metrics` (exposition Prometheus, hors OpenAPI). Authentification par jeton **JWT Bearer** unique (claims `sub`, `role`, `exp` — pas de refresh token séparé). Routeurs : `auth`, `health`, `history`, `predict`, `reports`, `users`.

Méthode	Chemin	Rôle requis	Description
GET	<code>/auth/me</code>	authentifié	Profil du compte courant (username, rôle)
POST	<code>/auth/refresh</code>	authentifié	Réémission du jeton avant expiration (même type de jeton)
POST	<code>/auth/token</code>	public (5/min)	Connexion par username + mot de passe → JWT
GET	<code>/health</code>	public	État de l'API (healthcheck Docker/Coolify)
POST	<code>/predict</code>	authentifié	Analyse d'une image (multipart, ≤ 5 Mo) → classe_predite + confiance
GET	<code>/predictions/history</code>	authentifié	Historique — agriculteur : ses prédictions ; admin : vue globale
GET	<code>/reports</code>	authentifié	Rapport d'entraînement (epochs, accuracy de validation)
GET	<code>/reports/evaluation</code>	authentifié	Détail d'évaluation du modèle (F1 par classe, etc.)
GET	<code>/reports/confusion-matrix</code>	authentifié	Matrice de confusion (image PNG)
GET	<code>/users</code>	admin	Liste des utilisateurs
POST	<code>/users</code>	admin	Création d'un membre — rôle imposé : agriculteur
DELETE	<code>/users/{user_id}</code>	admin	Suppression d'un compte agriculteur

Tableau vérifié contre le code des routeurs du dépôt (préfixes `/auth` et `/users`, routeur `users` entièrement en `Depends (verifier_role_admin)`). Le montage `/metrics` (Prometheus) s'ajoute à ces 12 opérations, hors `OpenAPI`.

5. Sécurité

- **JWT** signé (claims sub, role, exp) transmis en Authorization: Bearer — un seul type de jeton, réémis avant expiration via /auth/refresh.
- Mots de passe hachés **bcrypt** ; connexion par **username**, jamais par e-mail.
- **Rate limit 5/min** sur /auth/token (slowapi, par IP) + **blocage du compte après 5 échecs consécutifs** — deux mécanismes complémentaires (le rate limit par IP ne protège pas d'un attaquant lent ciblant un compte précis).
- **Upload plafonné à 5 Mo**, formats d'image contrôlés (validation du type MIME).
- Contrôle de rôle côté API (verifier_role_admin) en plus de la double barrière front.
- Référentiel couvert : **OWASP API Security Top 10, édition 2023**.

Choix assumé : le montage /metrics est **interne et non authentifié** — il n'est consommé que par Prometheus au sein du réseau Docker et n'expose aucune donnée personnelle (compteurs et histogrammes agrégés uniquement).

6. Données

PostgreSQL **asynchrone** (asyncpg + SQLAlchemy), migrations **Alembic** ; SQLite (aiosqlite) seulement en dev/préprod léger. Le schéma comporte **2 tables seulement** — le modèle IA n'est pas une table (checkpoint .pt chargé en mémoire, voir §3).

6.1 Table users

Colonne	Type	Contrainte / défaut
id	UUID	Clé primaire, défaut uuid4, indexée
username	String	Unique, non nul, indexée — identifiant de connexion
email	String	Unique, non nul (pas un identifiant de connexion)
hashed_password	String	Non nul — hachage bcrypt
role	String	Non nul, défaut « agriculteur » (ou « admin »)
created_at	DateTime	timezone=True, non nul, défaut now() UTC

6.2 Table predictions

Colonne	Type	Contrainte / défaut
id	Integer	Clé primaire, auto-incrémentée
user_id	UUID	Clé étrangère → users.id, non nul
nom_fichier	String	Non nul — l'image elle-même n'est jamais stockée
classe_predite	String	Non nul — une des 10 classes (ex. Tomato_Early_blight)
confiance	Float	Non nul — score 0.0 à 1.0
created_at	DateTime	timezone=True, non nul, défaut now() UTC

Cardinalité : un users possède 0..N predictions ; chaque prédiction appartient à exactement un utilisateur. Les noms de colonnes classe_predite / confiance sont les noms réels du schéma.

7. CI/CD

7.1 CI applicative — ci-app.yml

1. Audit de sécurité des dépendances : `pip-audit`.
2. Lint / format : `ruff`.
3. Service **PostgreSQL 16** + exécution des migrations **Alembic**.
4. pytest avec seuils de couverture : **API ≥ 75 %, modèle ≥ 80 %**.
5. Builds Docker de validation.

7.2 CI/CD modèle — ci-model.yml / cd-model.yml

Pipeline dédié au modèle : validation du checkpoint (`ci-model.yml`) puis déploiement du fichier `.pt` vers le volume du VPS (`cd-model.yml`, **dry-run par défaut**) — un nouveau checkpoint se déploie sans reconstruire l'image API.

7.3 Tests et qualité

Périmètre	Tests	Couverture
API	44	79,28 % (seuil 75 %)
Modèle	25	85,07 % (seuil 80 %)
Front	41	84 %
Base de données	2	—
Total	112	0 échec

Organisation agile : 41 pull requests fusionnées, branches `feature/*` et `fix/*`, pilotage par tableau GitHub Projects.

8. Monitoring et journalisation

L'API expose **4 métriques Prometheus** sur `/metrics`, séparant métriques applicatives et comportement du modèle :

Métrique	Signal
<code>tomatoscan_predictions_total</code>	Volume de prédictions (dont distribution des classes)
<code>tomatoscan_prediction_duration_seconds</code>	Latence d'inférence
<code>tomatoscan_prediction_confidence</code>	Confiance du modèle (détection de dérive)
<code>tomatoscan_errors_total</code>	Taux d'erreurs applicatif

- **Grafana** : dashboard de **7 panneaux** construit sur ces métriques (taux d'erreurs, temps de réponse, distribution des classes détectées, confiance moyenne du modèle... — intitulés exacts : voir `docs/monitoring.md`).
- **Alerte e-mail** configurée dans Grafana — seuil : `sum(rate(tomatoscan_errors_total[5m])) > 0,1` req/s pendant for: 1m ($\approx$ 1 erreur toutes les 10 s) ; code couleur : vert < 0,05, jaune 0,05–0,1, rouge > 0,1. Les 3 autres métriques sont surveillées visuellement, sans seuil (un seuil fixe universel produirait plus de faux positifs que de vrais signaux).
- **Journalisation** structurée avec **loguru** côté API.
- Un taux d'erreurs élevé et une dérive de confiance du modèle sont deux signaux distincts, traités séparément.

9. Déploiement

- **Conteneurisation Docker** — 5 services docker-compose : api, front, postgres, prometheus, grafana.
- Images publiées sur **ghcr.io** (GitHub Container Registry).
- Déploiement sur **VPS OVH (Roubaix)** orchestré par **Coolify**.
- Le checkpoint .pt est monté en **volume Docker en lecture seule** — un nouveau modèle se déploie sans reconstruire l'image API (voir §7.2).

Démarrage de l'API (lifespan) : 1. chargement de l'environnement — 2. création des tables si absentes (idempotent) — 3. bootstrap du compte admin (ADMIN_USERNAME / ADMIN_PASSWORD) — 4. chargement unique du modèle MobileNetV2 en mémoire. Ce chemin de démarrage dimensionne le start_period du healthcheck Docker.

Document généré depuis la source de vérité du dépôt (docs/modelisation.md, docs/architecture.md, README.md). Les données absentes de ces sources sont marquées [À COMPLÉTER].